

ULTRON - Intent Policy & Planner Architecture

Implementation handoff for integrating user-intent classification, execution policy, planning ownership, runtime authorization, and verification without disturbing existing working modules.

Status: Target architecture / implementation design. This document does not authorize implementation by itself; it defines the boundaries and invariants that implementation must preserve.

Primary goal: Fix the current architecture where task classification can turn a read-only investigation into DEBUGGING, the debugging planner assumes repair, and the verifier then correctly rejects the resulting plan because it expects execution.

1. Architectural Principle

Ultron must separate three concepts that are currently too tightly coupled:

TaskType - what kind of task the user is asking about.

UserIntent - what outcome the user wants.

ExecutionPolicy - what actions Ultron is actually authorized to perform.

The planner then determines **how** to achieve the authorized objective. It must not grant itself new authority.

TaskType != UserIntent != ExecutionPolicy

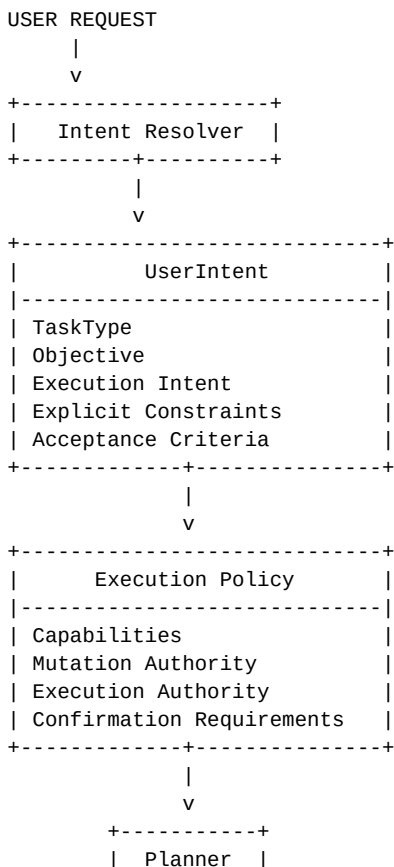
Task classification determines WHAT DOMAIN the task belongs to.

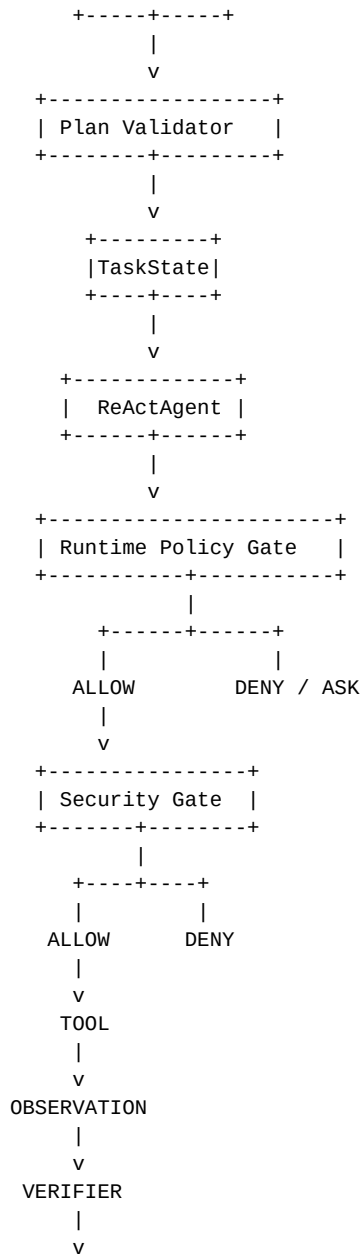
Intent determines WHAT OUTCOME the user wants.

Execution Policy determines WHAT ULTRON IS AUTHORIZED TO DO.

Planner determines HOW to accomplish the authorized goal.

2. Target Architecture





Goal + Policy + Acceptance Criteria

3. Ownership Boundaries

Component	Owns	Must NOT own
Intent Resolver	User goal, execution intent, explicit constraints, acceptance criteria	Tool execution or permission escalation
Task Classifier	Semantic task category such as DEBUGGING or RESEARCH	Deciding whether edits are authorized
Policy Resolver	Authoritative execution capabilities and mutation / confirmation rules	Choosing the task's business goal
Planner	Workflow / steps needed to satisfy the authorized goal	Granting itself edit / run / network authority
Plan Validator	Checks that plan actions fit ExecutionPolicy	Executing tools
TaskState	Canonical lifecycle state and transitions	Interpreting user intent
ReActAgent	Reasoning / orchestration	Bypassing policy
Runtime Policy Gate	Enforces policy before each action	Redefining user intent
Security Gate	Action-level security / confirmation	Changing task semantics
Verifier	Whether goal, requirements, policy and acceptance criteria were satisfied	Assuming DEBUGGING always means repair

4. UserIntent Contract

The Intent Resolver should produce one authoritative structured object. Other modules should consume this object rather than independently reinterpreting the original prompt.

```
UserIntent
■■■ raw_prompt
■■■ task_type
■■■ objective
■■■ execution_intent
■■■ explicit_requirements
■■■ constraints
■■■ acceptance_criteria
■■■ verification_strategy
```

Recommended execution intents:

- EXPLAIN - explain existing behavior or architecture.
- INSPECT - inspect/read the repository and report findings.
- DIAGNOSE - determine why a problem occurs without necessarily changing it.
- PLAN - investigate and produce a proposed implementation plan.
- REPAIR - diagnose and change the system to correct a problem.
- IMPLEMENT - build a requested feature/change.
- EXECUTE - perform an explicitly requested operational action.

5. Execution Policy

ExecutionPolicy is the authoritative capability contract derived from UserIntent. The planner receives it as an input and cannot broaden it.

```
ExecutionPolicy
■■■ allowed_capabilities
■   ■■■ READ_FILES
■   ■■■ SEARCH_REPOSITORY
■   ■■■ INSPECT_SYMBOLS
■   ■■■ GIT_READ
■   ■■■ WRITE_FILES
■   ■■■ DELETE_FILES
■   ■■■ RUN_COMMANDS
■   ■■■ RUN_TESTS
■   ■■■ NETWORK
■   ■■■ EXTERNAL_ACTIONS
■■■ mutation_authority
■   ■■■ FORBIDDEN
■   ■■■ ALLOWED
■   ■■■ REQUIRES_CONFIRMATION
■■■ execution_authority
■   ■■■ FORBIDDEN
■   ■■■ ALLOWED
■   ■■■ REQUIRES_CONFIRMATION
■■■ confirmation_requirements
```

6. Working Flow of the Intent Classifier / Resolver

```
1. RECEIVE RAW USER REQUEST
  |
  v
2. EXTRACT EXPLICIT LANGUAGE
  |
  |-- goal / desired outcome
  |-- action verbs
```

```

|-- prohibitions ("do not modify")
|-- permissions ("fix it", "run tests")
|-- constraints
|
V
3. CLASSIFY TASK DOMAIN
|
|-- DEBUGGING
|-- RESEARCH
|-- SOFTWARE_ENGINEERING
|-- CODE_REVIEW
|-- FILE_OPERATION
|-- SYSTEM_OPERATION
|-- ...
|
V
4. RESOLVE EXECUTION INTENT
|
|-- EXPLAIN
|-- INSPECT
|-- DIAGNOSE
|-- PLAN
|-- REPAIR
|-- IMPLEMENT
|-- EXECUTE
|
V
5. RESOLVE CONSTRAINTS / AUTHORITY
|
|-- read-only?
|-- edits allowed?
|-- commands allowed?
|-- confirmation required?
|
V
6. BUILD ACCEPTANCE CRITERIA
|
|-- what evidence proves the user's goal?
|-- what must NOT happen?
|
V
7. DERIVE EXECUTION POLICY
|
V
8. SEND UserIntent + Policy TO PLANNER

```

Important classifier rule

Explicit user constraints must survive classification. A phrase such as **"do not modify anything"** is not merely conversational text; it becomes an authoritative constraint that affects ExecutionPolicy and planning.

7. Concrete Examples

User request	TaskType	Intent	Policy	Expected plan
Explain how authentication works.	RESEARCH	EXPLAIN	READ_ONLY	Inspect -> trace -> explain
Investigate why login fails. Do not modify anything.	DEBUGGING	DIAGNOSE	READ_ONLY	Inspect -> trace -> diagnose -> report
Investigate the bug and propose a fix.	DEBUGGING	PLAN	PLAN_ONLY	Inspect -> diagnose -> design fix -> present plan
Investigate and fix the authentication bug.	DEBUGGING	REPAIR	EXECUTE	Reproduce -> diagnose -> edit -> test -> verify
Implement OAuth login.	SOFTWARE_ENGINEERING	IMPLEMENT	EXECUTE	Inspect -> design -> edit -> test -> verify

8. Planner Contract

The planner receives both `UserIntent` and `ExecutionPolicy`. Its output must be a plan whose actions are a subset of the authorized capabilities.

```
PlannerInput:
  UserIntent
  ExecutionPolicy
  Current TaskState
  Repository / Context information
```

```
PlannerOutput:
  TaskPlan
```

```
Invariant:
  every plan action MUST be authorized by ExecutionPolicy
  and MUST contribute to the user's stated objective.
```

For a read-only diagnosis, the planner must not inject steps such as "apply fix", "modify source", or "run regression tests" merely because `TaskType` is `DEBUGGING`.

9. Plan Validator

Plan validation is a structural safety check before execution. It catches planner mistakes early.

```
For each planned step:
  required_capabilities(step)
    |
    v
  compare with ExecutionPolicy
    |
    +-----+-----+
    |         |         |
  allowed   forbidden
    |         |
  accept    reject / replan
```

Examples:

- `READ_ONLY` + `write_file` -> reject before tool execution.
- `READ_ONLY` + `delete_file` -> reject before tool execution.
- `PLAN_ONLY` + run destructive command -> reject.
- `EXECUTE` + `write_file` -> potentially allowed, then still subject to Security Gate.

10. Runtime Policy Gate

The runtime must re-check authorization immediately before an action executes. This protects against model mistakes, planner mistakes, subagent behavior, stale plans, and future tool additions.

```
Model proposes action
  |
  v
Runtime Policy Gate
  |
  +-- not authorized --> DENY / ASK
  |
  +-- authorized -----> Security Gate
                          |
                          +--> DENY / ASK
                          |
                          +--> TOOL EXECUTION
```

This is the hard boundary. The model's system prompt is guidance; the runtime policy is enforcement.

11. Verifier Contract

The verifier should no longer infer completion requirements from TaskType alone. It should evaluate the combination of:

- UserIntent
- ExecutionPolicy
- TaskPlan
- TaskState / execution history
- Acceptance criteria
- Verification evidence

Example:

DEBUGGING + DIAGNOSE + READ_ONLY

Completion evidence:

- relevant code paths inspected
- failure mechanism identified
- evidence collected
- explanation delivered

No edit or test execution is required unless the user authorized it.

DEBUGGING + REPAIR + EXECUTE

Completion evidence:

- failure reproduced / sufficiently diagnosed
- authorized modification performed
- relevant tests or verification executed
- final behavior verified

12. What Must NOT Be Disturbed

- **Canonical TaskState:** keep it as the authoritative lifecycle state machine.
- **Event model / EventStore:** continue recording meaningful lifecycle and action events.
- **Context Manager / token budgeting:** do not duplicate context budgeting inside the new intent layer.
- **Repository Intelligence:** intent/policy should consume repository capabilities, not replace indexing, retrieval, RepoMap, or graph logic.
- **Model Router:** do not move model selection responsibilities into intent resolution.
- **Security scanners / security gate:** preserve them as the final action-level security boundary.
- **ReActAgent:** keep it focused on reasoning/orchestration; do not make it the source of truth for user authorization.
- **Existing working workflows:** changes should be additive and routed through existing interfaces wherever possible.

13. Migration Strategy

Phase A - Architecture mapping: inspect the current repository and map existing classifier, intent-understanding, planner, TaskState, ReAct, runtime, and security interfaces. Do not code yet.

Phase B - Introduce the contract: add the smallest canonical UserIntent + ExecutionPolicy representation without replacing working modules.

Phase C - Classification: make explicit user constraints such as read-only/no-modify first-class.

Phase D - Planner integration: make planning policy-aware and prevent unauthorized plan steps.

Phase E - Runtime enforcement: enforce policy before every action reaches a tool.

Phase F - Verifier integration: verify against intent/policy/acceptance criteria rather than crude TaskType assumptions.

Phase G - Production CLI validation: test real user workflows through the actual Ultron CLI and inspect runtime events/logs.

14. Required Test Matrix

Scenario	Expected policy	Expected behavior
Explain existing authentication flow	READ_ONLY	Read/search only; complete with explanation
Diagnose auth issue; do not modify	READ_ONLY	No edit/delete actions; diagnosis can complete
Diagnose and propose fix	PLAN_ONLY	Research + proposed plan; no mutation
Diagnose and fix auth issue	EXECUTE	Modification/testing allowed subject to security
Model attempts write during READ_ONLY	DENY	Runtime blocks action and records event
Planner emits unauthorized step	REJECT	Plan Validator rejects/replans
Security-sensitive command while authorized	ASK/DENY as applicable	Security Gate still applies

15. Non-Negotiable Invariants

- The planner cannot grant authority that the user did not provide.
- Explicit user constraints survive task classification and planning.
- READ_ONLY means mutation is blocked at the runtime action boundary, not merely discouraged by a prompt.
- TaskType.DEBUGGING does not imply permission to repair.
- Plan validation happens before execution.
- Runtime policy enforcement happens again immediately before tool execution.
- Security remains a separate final gate.
- Verifier completion criteria come from the user's actual intent and authorized workflow.
- Existing working modules retain ownership of their current responsibilities; do not create duplicate competing sources of truth.
- Every implementation change follows: Understand -> Architect -> Implement -> Test -> CLI -> Observe -> Fix -> CLI -> Prove -> Move Forward.

16. Implementation Handoff Summary

The immediate implementation objective is **not** to rewrite Ultron's planner, verifier, ReAct loop, or security system. It is to introduce a clean intent/policy contract and connect existing components to that contract while preserving their existing responsibilities.

The first coding step should therefore be a repository-level architecture audit that maps this target design to the actual current interfaces and identifies the smallest ownership-safe integration points.

Success condition: a real Ultron CLI request such as "Investigate why authentication fails. Do not modify anything." must remain a read-only diagnostic workflow all the way through planning, ReAct, runtime authorization, tools, and verification - with no accidental repair plan and no mutation.

Prepared as an implementation architecture handoff for Project Ultron.